

Computer Networks Lab (Assignment 1) — Comprehensive Viva & Concept Cheat Sheet

Student: Mandeep Shaw

Roll Number: 302510501002 | **Section:** A1

Course: BCSE-III Computer Networks Laboratory, Jadavpur University

Topic: Error Detection Schemes in Network Transmission (16-bit Internet Checksum vs CRC-8, 10, 16, 32)

High-Level Pitch (Say this if the Professor asks: "Tell me in 2 minutes what you did")

"In this assignment, I designed and implemented a client-server socket pipeline in C to transmit 64-byte Ethernet-like frames across localhost. I compared two major error-detection families: the **16-bit Internet Checksum** (RFC 1071, used in TCP/IP) and **Cyclic Redundancy Checks** (CRC-8, CRC-10, CRC-16, and CRC-32 IEEE 802.3).

I subjected both algorithms to 1,000,000 experimental transmission trials across five distinct error patterns (single-bit, double-bit, odd-parity, burst, and random errors), backed by 5,000,000 micro-benchmark iterations.

The fundamental engineering takeaway is the **Speed vs. Reliability Tradeoff**: The 16-bit Checksum runs at **1,907 MB/s** (~20× faster than bit-serial CRC) because it processes 16-bit words directly in CPU registers, but it is vulnerable to compensating errors (~2.08% miss rate on double-bit errors). Conversely, CRC provides mathematical guarantees: 100% detection of all single-bit, double-bit (for $r \geq 10$), odd-parity, and burst errors of length $\leq r$, but bit-serial software division runs at ~95 MB/s. This explains why **TCP/UDP in the OS kernel use Checksum**, while **Ethernet/Wi-Fi hardware NICs use CRC-32**."

Section-by-Section Report Walkthrough

1. Frame Structure (Section 3.1)

Each transmitted frame is **64 bytes**, structured like an Ethernet frame: +-----+-----+-----+-----+
+-----+-----+-----+-----+ | Preamble (8 bytes) | Dest MAC (6 bytes) | Src MAC (6 bytes) | Type/
Length (2 bytes) | Payload (38B) | +-----+-----+-----+-----+
+-----+-----+-----+-----+ Total Protected Header + Payload = 60 bytes (indices 0 to 59). Trailing
4 bytes = Frame Check Sequence (FCS / CRC-32 / Checksum / Padding). - **Control Handshake (3 bytes)** sent before the frame: -
frameInfo[0]: Scheme selector (1: Checksum, 2: CRC-8, 3: CRC-10, 4: CRC-16, 5: CRC-32). Bit 7 is set if bypass mode is enabled. - frameInfo[1]: Error
injection flag (0: Clean, 1: Injected). - frameInfo[2]: Error model (1: Single, 2: Double, 3: Odd, 4: Burst, 5: Random).

2. The 16-Bit Internet Checksum (RFC 1071)

How it works:

- Divide data into consecutive **16-bit words** (big-endian network order).
- Sum words using **one's complement addition**:
 - Add into a 32-bit unsigned accumulator.
 - Fold carries back into the lowest 16 bits (**end-around carry**): while (sum > 0xFFFF) sum = (sum & 0xFFFF) + (sum >> 16);
- Take the bitwise **one's complement inversion** (~sum & 0xFFFF).
- Verification at Receiver**: Sum all words including the received checksum word. If the transmission is error-free, the resulting 16-bit one's complement sum must be 0xFFFF (or inverted to 0x0000).

Critical Bug We Fixed in the Starter Code:

- The Bug**: The initial code was performing a simple 8-bit byte-by-byte sum (uint8_t), completely violating RFC 1071!
- The Fix**: Rewrote checksum.c to accumulate pairs of bytes as 16-bit words ((data[i] << 8) | data[i+1]) with end-around carry folding.

3. Cyclic Redundancy Check (CRC) Principles

How it works:

- Treats the message bit-stream as a polynomial $M(x)$ with coefficients in $\mathbb{GF}(2)$ (arithmetic modulo 2, where addition and subtraction are both bitwise **XOR**).
- Appends r zeros to the message: $M(x) \cdot x^r$.
- Divides by a standard **Generator Polynomial** $G(x)$ of degree r : $\frac{M(x) \cdot x^r}{G(x)} = Q(x) + \frac{R(x)}{G(x)}$
- The remainder $R(x)$ (degree $< r$) is the **CRC FCS**.
- Transmitted codeword: $T(x) = M(x) \cdot x^r \oplus R(x)$.
- Receiver Check**: Divides received polynomial by $G(x)$. If remainder is 0, frame is valid.

Evaluated Polynomial Standards (Table 2):

Standard	Degree r	Polynomial $G(x)$	Hex Code	Typical Real-World Application
CRC-8	8	$x^8 + x^7 + x^6 + x^4 + x^2 + 1$	0x1D5	Bluetooth, ATM header HEC, Sensor nets
CRC-10	10	$x^{10} + x^9 + x^5 + x^4 + x + 1$	0x633	ITU-T I.432.1 ATM OAM cells

Standard	Degree	\$r\$ Polynomial	\$G(x)\$	Hex Code	Typical Real-World Application
CRC-16	16	$x^{16} + x^{15} + x^2 + 1$	(CRC-16-ANSI)	0x18005	USB packets, Modbus, Bisync
CRC-32	32	IEEE 802.3 Ethernet Standard		0x104C11DB7	Ethernet, Wi-Fi 802.11, PNG, ZIP, IPv4

🔗🔗 Critical Bug We Fixed in the Starter Code:

- **The Bug:** `crc.c` checked if `remainder == 0` or skipped bit shifts if data bits were 0, corrupting modulo-2 polynomial division for non-multiple-of-8 degrees (especially CRC-10 and CRC-32).
- **The Fix:** Implemented standard bit-serial long division: shift in bits MSB-to-LSB, and if the top bit `$@` is 1, XOR with the generator polynomial. Then flush with `$r$` trailing zero bits.

🔗🔗 Summary of Empirical Results (Memorize These Numbers!)

Across **1,000,000 total simulation trials** (200,000 trials per error model across 5 schemes) and **5,000,000 benchmark iterations**:

Error Model	Checksum Detection	CRC-8 Detection	CRC-10 Detection	CRC-16 Detection	CRC-32 Detection
Single-Bit	100.0000%	100.0000%	100.0000%	100.0000%	100.0000%
Double-Bit	97.9175% (Miss: 2.08%)	99.8035% (Miss: 0.20%)	100.0000%	100.0000%	100.0000%
Odd-Parity	99.9365% (Miss: 0.063%)	100.0000%	100.0000%	100.0000%	100.0000%
Burst (\$L=12\$)	99.6730%	99.6730% (Bound: 99.61%)	99.9225% (Bound: 99.90%)	100.0000%	100.0000%
Burst (\$L=32\$)	99.6140%	99.6140%	99.8970%	99.9985%	100.0000%
Random Errors	99.9985%	99.6080%	99.9025%	99.9985%	100.0000%

Computational Benchmark (5,000,000 frames):

- **16-Bit Checksum:** 30.00 ns / frame $\rightarrow$ 1,907.14 MB/s (Baseline $1.00\times$)
- **CRC-8:** 569.26 ns / frame $\rightarrow$ 100.52 MB/s ($18.97\times$ slower)
- **CRC-10:** 573.40 ns / frame $\rightarrow$ 99.79 MB/s ($19.11\times$ slower)
- **CRC-16:** 587.24 ns / frame $\rightarrow$ 97.44 MB/s ($19.57\times$ slower)
- **CRC-32:** 629.36 ns / frame $\rightarrow$ 90.92 MB/s ($20.98\times$ slower)

🔗🔗 The 3 Concrete Boundary Collision Cases (Section 7)

Be prepared to explain these three exact cases on a whiteboard:

Case 1: Both Schemes Detect (Standard Corruption)

- **Corrupted Bytes:** Offset 11 (0x23 $\rightarrow$ 0x27, bit flipped) and Offset 33 (0x6E $\rightarrow$ 0x6F).
- **Checksum:** Changed from 0x5DEC to 0x5E10 $\rightarrow$ **DETECTED**.
- **CRC-8:** Changed from 0xFF to 0x2D $\rightarrow$ **DETECTED**.

Case 2: CRC-8 Collision (CRC Misses, Checksum Detects!)

- **What happened:** Flipped bits at payload offset 11 and offset 23 (12 bytes / 96 bits apart).
- **The Math:** The error pattern forms an error polynomial $E(x) = x^{96} + 1$. Because $E(x)$ is an exact algebraic multiple of $G(x) = x^8 + x^7 + x^6 + x^4 + x^2 + 1$, the division yields remainder $R(x) = 0$!
- **CRC-8 Result:** Remainder remained 0xFF $\rightarrow$ **MISSED (Undetected corruption)**.
- **Checksum Result:** Changed from 0x5DEC to 0x5DFE $\rightarrow$ **DETECTED**.

Case 3: Compensating Sum Collision (Checksum Misses, CRC-8 Detects!)

- **What happened:**
 - Byte offset 29 decreased by 4: 0x6C ('l') $\rightarrow$ 0x68 ('h') [$\Delta = -4$].
 - Byte offset 41 increased by 4: 0x61 ('a') $\rightarrow$ 0x65 ('e') [$\Delta = +4$].
- **The Math:** In the 60-byte frame, Byte 29 is at serialized index 45, and Byte 41 is at serialized index 57. Both are the least-significant bytes (LSB) of 16-bit words. When summed: $\Delta_{\text{sum}} = (-4) + (+4) = 0$!
- **Checksum Result:** Net sum is unchanged (0x5DEC $\rightarrow$ 0x5DEC) $\rightarrow$ **MISSED (Blind to error!)**.
- **CRC-8 Result:** Remainder changed from 0xFF to 0xBE $\rightarrow$ **DETECTED**.

🔗🔗 Top 10 Professor Viva Questions & Bulletproof Model Answers

Q1: “Why do we add the carry bit back into the sum in One’s Complement Checksum (End-Around Carry)?”

Answer: “In one’s complement representation, negative numbers are formed by bitwise inversion. A mathematical property of one’s complement addition modulo $2^{16} - 1$ is that any overflow carry out of the most significant bit (bit 16) has the modular weight of 1 . Adding it back to the least significant bit (end-around carry) ensures that the checksum is completely independent of the endianness (byte-

ordering) of the host architecture. Whether an architecture sums 16-bit words in big-endian or little-endian, the resulting checksum byte sequence is identical after byte-swapping.”

Q2: “Why does CRC detect 100% of odd-numbered bit errors?”

Answer: “Because the generator polynomial $G(x)$ contains $(x + 1)$ as a factor (i.e., $G(1) = 0$). Any odd number of bit flips corresponds to an error polynomial $E(x)$ with an odd number of terms, which means evaluating $E(1) = 1 + 1 + \dots + 1 \pmod 2 = 1 \neq 0$. Therefore, $E(x)$ can never be divisible by $(x + 1)$, and consequently can never be divisible by $G(x)$. Thus, the remainder can never be zero, guaranteeing 100% detection.”

Q3: “What is the theoretical burst error detection capability of CRC?”

Answer: “If a burst of noise flips bits over a span of length L : 1. For $L \leq r$ (where r is the polynomial degree): Detection is **strictly 100%**, because the error polynomial $E(x) = x^k \cdot B(x)$ has degree $< r$, making division by $G(x)$ impossible without a remainder. 2. For $L = r + 1$: The miss rate is exactly $(\frac{1}{2})^{r-1}$. 3. For $L > r + 1$: The miss rate is bounded by $(\frac{1}{2})^r$. For CRC-32, $(\frac{1}{2})^{32} \approx 2.33 \times 10^{-10}$, meaning less than 1 in 4 billion bursts can escape detection.”

Q4: “If CRC-32 is mathematically superior, why does TCP still use a 16-bit Checksum?”

Answer: “It is a deliberate protocol layer engineering tradeoff: - **Transport Layer (TCP/UDP)** runs in **software inside the OS kernel on the host CPU**. The 16-bit checksum requires simple ALU integer additions ($\sim 1,907 \text{ MB/s}$), taking minimal CPU cycles per packet. - **Data Link Layer (Ethernet/Wi-Fi)** runs in **dedicated hardware (NIC ASICs / FPGAs)**. Hardware can compute CRC-32 concurrently at full line rate using simple linear feedback shift registers (LFSR) with zero CPU overhead. Since Ethernet’s CRC-32 already filters out 99.999999% of transmission noise at Layer 2, TCP’s lightweight checksum primarily serves as an end-to-end sanity check against memory corruption inside intermediate routers.”

Q5: “Can you give me an example where the Internet Checksum fails completely?”

Answer: “Yes! Two common failure modes: 1. **Compensating Errors:** If one byte increases by K and another byte in the corresponding alignment decreases by K , the sum cancels out ($\Delta \text{Sum} = 0$). In our tests, double-bit errors missed 2.08% of corruptions. 2. **Byte Swapping / Transposition:** If two 16-bit words swap positions, the commutative property of addition leaves the sum unchanged. Checksum cannot detect reordered words.”

Q6: “Why did CRC-8 miss 0.20% of double-bit errors while CRC-10, 16, and 32 caught 100%?”

Answer: “To detect all double-bit errors, the generator polynomial $G(x)$ must not divide $(x^k + 1)$ for any $k \leq$ frame length. The period (maximum cycle length) of CRC-8’s polynomial is only 127 bits. Since our frame is 60 bytes (480 bits), $480 > 127$. Multiple pairs of bit flips 127 bits apart create an error polynomial divisible by CRC-8. For CRC-10, CRC-16, and CRC-32, the periods far exceed 480 bits, guaranteeing 100% double-bit error detection.”

Q7: “How is CRC implemented in production software to achieve gigabyte-per-second speeds?”

Answer: “Production software uses a **Table-Driven (Lookup Table) approach**. Instead of an inner loop doing 8 bit-shifts and XORs per byte, a 256-entry table of precomputed 32-bit remainders is used: `c_crc = (c_crc >> 8) ^ crc32_table[(c_crc ^ byte) & 0xFF]`; This processes 8 bits in a single memory lookup and XOR. Furthermore, modern CPUs provide hardware instructions like `_mm_crc32` (Intel SSE4.2 / ARMv8) which process 64 bits per CPU cycle, achieving over 20 GB/s.”

Q8: “What socket system calls did you use for the pipeline?”

Answer: “We used standard BSD sockets: - **Server (receiver .c):** `socket(AF_INET, SOCK_STREAM, 0)` `bind()` to port 5000 `listen()` `accept()` client connection `recv()` in a loop. - **Client (sender .c):** `socket()` `connect()` to 127.0.0.1:5000 `inject error` `send()` 3-byte control header `send()` 64-byte frame.”

Q9: “What is Galois Field GF(2) and why do we use it in CRC?”

Answer: “GF(2) is the finite Galois Field of two elements: $\{0, 1\}$. Addition and subtraction in GF(2) are identical and correspond to the exclusive-OR (XOR) operation without carries. This makes modulo-2 polynomial arithmetic trivial to implement in digital circuits and software, as no carry propagation delay is required.”

Q10: “If an error is detected, what should the networking protocol do?”

Answer: “In our evaluation script, corrupted frames were logged and dropped. In a production protocol like TCP or HDLC, error detection is coupled with **ARQ (Automatic Repeat reQuest)**: - The receiver sends a NACK (Negative Acknowledgment) or drops the frame. - The sender’s

retransmission timer expires, triggering automatic retransmission (e.g., Stop-and-Wait, Go-Back-N, or Selective Repeat)."

💡 Quick Tips for Viva Day

1. **Always mention RFC 1071** when talking about Checksum (professors love RFC citations).
2. **Mention $G(x) = 0 \pmod{x+1}$ factor** when asked about odd-parity detection.
3. **Mention $L \leq r$ guaranteed detection** and $(\frac{1}{2})^r$ bound when asked about burst errors.
4. **Point to the 1,907 MB/s vs 100 MB/s measurement** to justify why TCP uses Checksum and Ethernet uses CRC.